InventIQ

Requirements Specification & Architectural Design Deliverable

Deliverable 1

Team Member	Student ID / Email	Role (optional)
Luis Alejandro Duarte Daza	luduarte@unal.edu.co	
Santiago Andres Amezquita Becerra	samezquitab@unal.edu.co	
Miguel Angel Suarez Montiel	migsuarezmo@unal.edu.co	
Sebastian Camilo Parra Siabato	separras@unal.edu.co	

Course / Section	Software Engineering I
Instructor	Magda Lucia Mejia Torres
Submission Date	05/10/2026
Repository URL	https://github.com/luduarte23/Gestor-Negocios

0. Revision History

Version	Date	Author	Description of Changes
0.1	28/09/26	Luis Alejandro Duarte Daza	Copy of the template and general adjustments to the organization of the document in line with the project and the software development life cycle.
0.2	30/09/26	Luis Alejandro Duarte Daza	introduction and initial definition of the in-scope, out-scope
0.3	1/10/26	Santiago Andres Amezquita Becerra	User stories design and adjusting of requirements based on the insight gained by them
0.4	2/10/26	Miguel Angel Suarez Montiel	Adjustment of the system scope. Definition of stakeholders and user roles. Development of risks and assumptions
0.5	3/10/26	Sebastian Camilo Parra Siabato	Elaboration of Section 3: UML component diagram (with public interfaces per module) and UML deployment diagram (with protocols on every connection), together with their explanations. Module 06 "Infrastructure and Deployment" was added in Section 4, and "System Core" was replaced by MOD-06 in the traceability of NFR-05.
1.0	4/10/26	Santiago Andres Amezquita Becerra	Elaboration of final conclusions and last check

Version	Date	Author	Description of Changes
			of compliance with project guidelines

1. System Overview and Scope

1.1 System Purpose

Owners of small retail businesses and store managers spend hours every week manually checking inventory levels, calculating restocking quantities, and estimating when to place orders with suppliers. Currently, these critical operational decisions rely on intuition, manual spreadsheets, or error-prone handwritten lists, which leads to financial losses caused by expired products, stockouts of popular items, and uncoordinated supplier orders that fail to take advantage of volume discounts or free-shipping thresholds.

The main objective of InventIQ is to act as an automated operational decision engine that eliminates manual estimation in inventory management for small businesses. Unlike a passive data-recording tool, the system captures sales in real time through barcode scanning with FIFO inventory deduction, and continuously analyzes sales history, supplier lead times, available cash flow, and batch expiration dates. With this information, it autonomously determines the optimal reorder points (ROP), prioritizes purchases under budget limits, and calculates dynamic discount scales for merchandise close to expiring, maximizing the profitability of the business.

1.2 System Scope

In scope for this version:

- **Real-time sales recording and barcode scanning:** Sales capture module accelerated by a USB barcode scanner (*keyboard wedge*), automatically deducting units from the oldest inventory batch according to the FIFO rule.
- **Transaction history (Sales and Purchases):** Recording and storage of the daily operational history as a database for sales velocity calculations and turnover analysis.
- **Dynamic Reorder Point calculation:** Continuous determination of when and exactly how much to buy for each product, analyzing historical sales velocity, supplier lead times, and safety stock.
- **Supplier order consolidation and drafts:** Automatic grouping of low-stock products by supplier, generating structured purchase orders and editable message drafts.
- **Minimum order optimization per supplier:** Algorithmic selection of high-turnover products to reach the minimum amount or volume required by the supplier to obtain free shipping or volume discounts.
- **Purchase prioritization under a budget limit:** Decision engine that filters and ranks purchase orders when the total cost exceeds the available weekly cash flow, prioritizing items with higher margin and turnover.
- **Dynamic discount management by expiration:** Automatic application of progressive discount scales to perishable batches according to their expiration date and the current sales pace.
- **Direct integration with external messaging APIs (WhatsApp):** Orders and messages are generated on screen, with an option for them to be delivered to a WhatsApp account belonging to the administrator.
    - In the context of the course, a sandbox will be used for the implementation. In a real production case, it is only necessary to start paying for the required subscription and adjust settings in the service account, with no changes to the Java code

Out of scope for this version:

- **Complex Point of Sale (POS) peripheral hardware:** Thermal receipt printers, automatic cash drawers, and card readers/electronic payment terminals.
- **Customer credit module and risk assessment ("Fiado"):** The assessment of credit limits and payment history for deferred sales is postponed to future versions.
- **Multi-branch and multi-warehouse management:** The system manages a single inventory and does not handle transfers between different locations or warehouses.
- **Native mobile applications and offline operation:** The system works exclusively as a responsive web application rendered on the server.
- **Alternative product identification mechanism**: The system works with products that have a standardized barcode.

1.3 Stakeholders and User Roles

Instructions: List every actor that interacts with the system, human or automated (e.g., a scheduled job, an external API). One row per role.

Role	Description	Key Goals / Interests
Administrators	Store owner or manager. Access the dashboard, set up complete supplier information, and configure relevant details regarding inventory thresholds and prices.	Reduce losses due to product expiration. Receive information and alerts without having to constantly check the system
Cashiers	Store clerk. Use the sales registration screen to scan products using the barcode or, if necessary, manually enter the product number.	Be able to record sales quickly, without too much hassle, and without requiring a lot of time for training
Generic Barcode Scanners	Optional barcode scanner belonging to the store. Reads the physical barcode and inputs it as text in a form field	The system must be configured to receive text input from a generic barcode scanner and place it in the appropriate field. Manual input should also be an option.
Twilio (WhatsApp API)	Notifies administrators with relevant system information. Receives system information, formats it into preconfigured messages, and sends it to the WhatsApp account provided by the administrator.	Allow the administrator's WhatsApp account to be entered into the system to send the information, confirming a successful connection. Receive the information in the correct format for sending messages

2. Software Requirements Specification (SRS)

2.1 High-Level Functional Requirements (FR)

ID	Requirement	Priority
FR-01	The system will allow users to log in and will restrict access to functions according to the assigned role	Mandatory
FR-02	The system will record sales by scanning USB barcodes (keyboard wedge) or by manual entry and will automatically deduct units from the oldest batch following the FIFO rule.	Mandatory
FR-03	The system will automatically calculate the Reorder Point (ROP) for each product by analyzing daily sales history, lead times, and safety stock.	Mandatory
FR-04	The system will group low-stock products by supplier and generate a draft purchase order with editable quantities.	Mandatory
FR-05	The system will suggest including additional high-turnover products to reach the Minimum Order Quantity (MOQ) required for free shipping or a volume discount.	Mandatory
FR-06	The system will filter and prioritize purchase orders according to profit margin and turnover when the total cost exceeds the available weekly cash budget	Mandatory
FR-07	The system will calculate and apply dynamic discount scales to batches of perishable products, considering their expiration date and sales velocity	Mandatory
FR-08	The system will allow the administrator to manually adjust operational parameters (stock levels, lead times, budget limits)	Mandatory
FR-09	The system will generate a structured message proposal with the order summary for manual or automatic sending to the supplier.	Desirable
FR-10	The system will record the confirmation of receipt of purchase orders and will automatically update the available inventory.	Mandatory
FR-11	The system will keep an audit log of the automated decisions executed by the replenishment engine.	Optional

2.2 Non-Functional Requirements (NFR / Quality Attributes)

ID	ISO 25010 Category	Requirement	Metric / Target
NFR-01	Performance Efficiency	The system will run the full inventory analysis, ROP calculation, and budget prioritization within optimal response times	Execution time $< 1.5s$ for the analysis of catalogs of up to 1,000 products.
NFR-02	Usability	The system will allow the administrator to review, edit, and approve a generated purchase order with a minimum number of user interactions.	Order approval completed in ≤ 3 clicks from the main dashboard.
NFR-03	Security	The system will restrict administrative functions (stock adjustments, budget, and order approval) to users authenticated as administrator.	100% of administrative endpoints protected through session-based authentication.
NFR-04	Maintainability	The business domain logic (ROP calculation, MOQ optimization, discounts, and budget) will be isolated from the infrastructure.	>= 80% code coverage in unit tests running without a web server or database.
NFR-05	Portability	The system will operate as a self-contained web service that runs with a single command using Java 21 and an embedded database using the architecture of spring boot with an embedded server.	100% executable without preinstalled dependencies or servers

2.3 Detailed Functional Scenarios

Option A - User Stories with Acceptance Criteria (Agile Approach)

US-01: [Generate draft purchase orders]

- **As an** administrator, **I want to** group products below their reorder point into a draft order for each supplier, **So that** I can check and send the order without checking the stock manually.

Acceptance Criteria (Gherkin):

Scenario 1 - Successful execution path

- **Given** The store has 6 products under the reorder point, 4 from supplier A and 2 from supplier B,
- **When** the reorder analysis executes,
- **Then** the administrators get one draft order per supplier, with the products and suggested quantities editable by the user and the total cost.

Scenario 2 - Edge case or error handling

- **Given** A product or products is under the reorder point but it has no assigned supplier,
- **When** the reorder analysis executes,
- **Then** the system groups the unassigned products and marks them as to be reviewed.

US-02: [Complete minimum supply order (complements US-01)]

- **As an** administrator, **I want to** have automatic suggestions of additional high rotation products when a purchase order falls short of the minimum order amount, **So that** I can get free shipping or discounts for buying in volume.

Acceptance Criteria (Gherkin):

Scenario 1 - Successful execution path

- **Given** a supplier requires a minimum order of 500.000 COP for free shipping and the draft order is 400.000 COP,
- **When** the administrator opens the order,
- **Then** the system suggests high selling products to get the order up to the minimum order, shows the total after the suggestions and lets the administrator accept or modify the order.

Scenario 2 - Edge case or error handling

- **Given** a supplier requires a minimum order of 500.000 COP for free shipping, but the budget is 400.000 COP and the draft equal to the budget,
- **When** the system tries to modify the order,
- **Then** it doesn't add products and gives the option of continuing the purchase with the draft or deferring the purchase for the next order time.

US-03: [barcode scanning and automated inventory]

- **As a** cashier, **I want to** scan a products barcode and have the sale recorded and the stock updated instantly,
- **So that** I can make the selling process faster and have an accurate inventory system that's automated.

Acceptance Criteria (Gherkin):

Scenario 1 - Successful execution path

- **Given** a product with 6 units in stock 3 that expire on 08/10/2026 and 3 that expire on 30/10/2026,
- **When** the cashier scans a product with the expiration date on 08/10/2026 two times,
- **Then** the stock of the product that expires on that date is deducted by 2 leaving it with 1 unit.

Scenario 2 - Edge case or error handling

- **Given** the scanned barcode is not in the system,
- **When** the cashier scans the barcode,
- **Then** the system rejects the product with a message, the stock is not modified and no sale is recorded letting the cashier continue.

US-04: [dynamic discounts]

- **As an** administrator, **I want to** have discounts on products that are close to expiring taking into account the rate that that specific product sells at, **So that** the store doesn't lose money by having products expire on the shelf and then thrown out.

Acceptance Criteria (Gherkin):

Scenario 1 - Successful execution path

- **Given** perishable product batch A of 15 is in stock the expiration date of product batch A is 10 days away and it sells at average once each day so there are 5 at risk of expiring,
- **When** the daily expiring product check runs,
- **Then** the system puts a discount based on the time of expiring and the rate of selling of the product.

Scenario 2 - Edge case or error handling

- **Given** a product B has no selling history and the expiration date of a product B batch is 10 days away,
- **When** the daily expiring product check runs,
- **Then** the system sends an alert to the administrator asking for a manual discount or to continue without it.

US-05: [confirm order arrival and update stock]

- **As an administrator**, **I want to** confirm the arrival of a purchased order and have the inventory automatically update, **So that** stock is accurate and it saves the manual work of updating it by hand.

Acceptance Criteria (Gherkin):

Scenario 1 - Successful execution path

- **Given** an order arrived with 50 units of product A with expiration date to 20/12/2026,
- **When** the administrators confirms the arrival of the correct order in product and amount,
- **Then** the system adds the batch of 50 units of product A to the inventory with its respective expiration date and marks the order as received.

Scenario 2 - Edge case or error handling

- **Given** a purchase order for 50 units of product P,
- **When** the administrator confirms receipt of only 30 units,
- **Then** the system adds exactly 30 units to stock, marks the order as "partially received" with 20 units pending, and keeps those 20 visible so the administrator can wait for them or close the order.

3. Architectural Design

3.1 Architectural Style Selection & Rationale

Selected Architectural Style:

Layered (N-Tier) & MVC

Technical Rationale:

The project adopts a Layered architecture structured under the Model-View-Controller (MVC) design pattern. This choice responds directly to the functional requirements, quality attributes, and constraints of the project, organizing the software's responsibilities into four decoupled layers:

1. **Layer Isolation and Pure Business Domain (NFR-04, FR-02 to FR-07):** The architecture isolates the core business logic in an independent domain layer, responsible for replenishment calculations, inventory deduction rules, grouping of supplier orders, minimum purchase thresholds, budget prioritization, and expiration discounts. By decoupling these operational rules from the presentation and storage infrastructure, isolated unit tests can be run

without needing to start web services or databases, ensuring the required code coverage (NFR-04) over the decision logic.

2. **In-Memory Processing and Low Latency (NFR-01, NFR-02, FR-02, FR-09):** A server-rendered monolithic structure eliminates distributed network latency and the communication overhead between independent services. All data processing happens in memory within a single execution unit connected to an embedded database. This ensures that barcode sales recording is processed without interface delays (NFR-02), and that the full inventory analysis runs within the required response times (NFR-01) to generate purchase drafts and message proposals for suppliers (FR-09).
3. **Centralized Security and Access Control (NFR-03, FR-01, FR-08):** The security mechanisms intercept requests at the boundary of the presentation layer, maintaining secure user sessions managed by the server. This guarantees role-based access control between administrators and cashiers (FR-01), protecting the administrative endpoints (NFR-03) for inventory adjustments, parameter configuration, and budget assignment (FR-08).
4. **Transactional Consistency and Portability (NFR-05, FR-10, FR-11):** Managing purchase receipts (FR-10) and the audit log of automated decisions (FR-11) within a single database boundary ensures the transactional consistency of the information. Likewise, packaging the application components and the embedded database into a single deployment unit guarantees full portability, allowing the system to run in any environment without requiring preinstalled external databases (NFR-05).

Accepted Compromise (*Trade-off*):

- **Horizontal Scalability vs. Architectural Simplicity:** Choosing a monolithic architecture sacrifices the ability to scale components independently. However, this compromise is accepted because the operational scope is focused on single-branch businesses. The simplicity of deployment and the elimination of failures in distributed systems outweigh the need for microservice-level scalability.
- **Technological Coupling vs. Development Speed:** All layers share the same technology platform. Although this limits the inclusion of components developed in other languages, it maximizes the team's development speed and eliminates the need for data translation protocols between processes.

3.2 Development / Component View (UML Diagram)

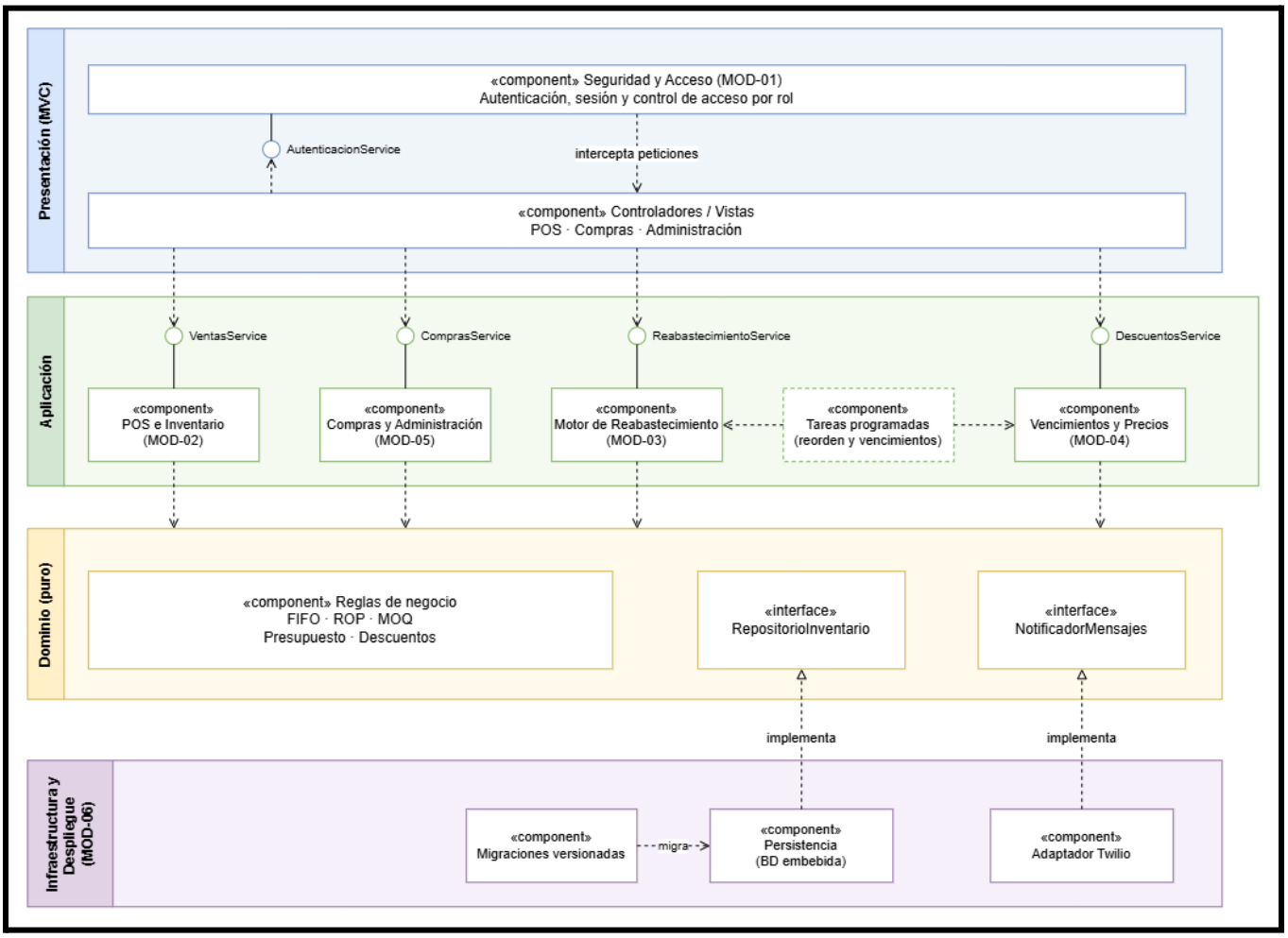


Diagram Explanation:

The system is organized into four decoupled layers, and dependencies always point downward. Each module exposes a public interface, and the other components only know that interface, never its internal implementation.

Presentation (MVC). Contains the controllers and views for the POS, purchasing, and administration. The Security and Access module (MOD-01) intercepts every request at this boundary and validates the session and the role before it reaches the logic. It exposes AutenticacionService, which the controllers use for login (FR-01, NFR-03).

Application. Brings together the modules that orchestrate the use cases, each with its public interface:

- POS and Inventory (MOD-02), with VentasService: records sales by barcode and deducts batches using FIFO (FR-02).
- Replenishment Engine (MOD-03), with ReabastecimientoService: calculates the ROP, groups by supplier, optimizes the MOQ, and prioritizes under budget (FR-03 to FR-06).
- Expirations and Pricing (MOD-04), with DescuentosService: calculates dynamic discounts by expiration (FR-07).
- Purchasing and Administration (MOD-05), with ComprasService: manual adjustments, receipt of orders, messages to suppliers, and auditing (FR-08 to FR-11).

The scheduled tasks run the reorder analysis and the daily expiration review by invoking those same services, with no user intervention.

Domain. Concentrates the pure rules (FIFO, ROP, MOQ, budget, and discounts) and does not depend on any other layer. It only defines the interfaces it needs toward the outside: RepositorioInventario and NotificadorMensajes. For this reason, it can be tested with unit tests without a web server or database (NFR-04).

Infrastructure and Deployment (MOD-06). Implements those interfaces: persistence over the embedded database, the versioned schema migrations, and the Twilio adapter. It also groups the configuration and packaging that make the system portable (NFR-05).

Decoupling logic.

- Controllers depend on the modules' interfaces and not on their code, so a module's implementation can change without affecting the presentation.
- The infrastructure depends on the domain (it implements its interfaces), not the other way around. The database or the messaging provider can be changed without touching the business rules.
- A Twilio outage does not block the main flow, because the Application module only knows NotificadorMensajes.

3.3 Physical / Deployment View (UML Diagram)

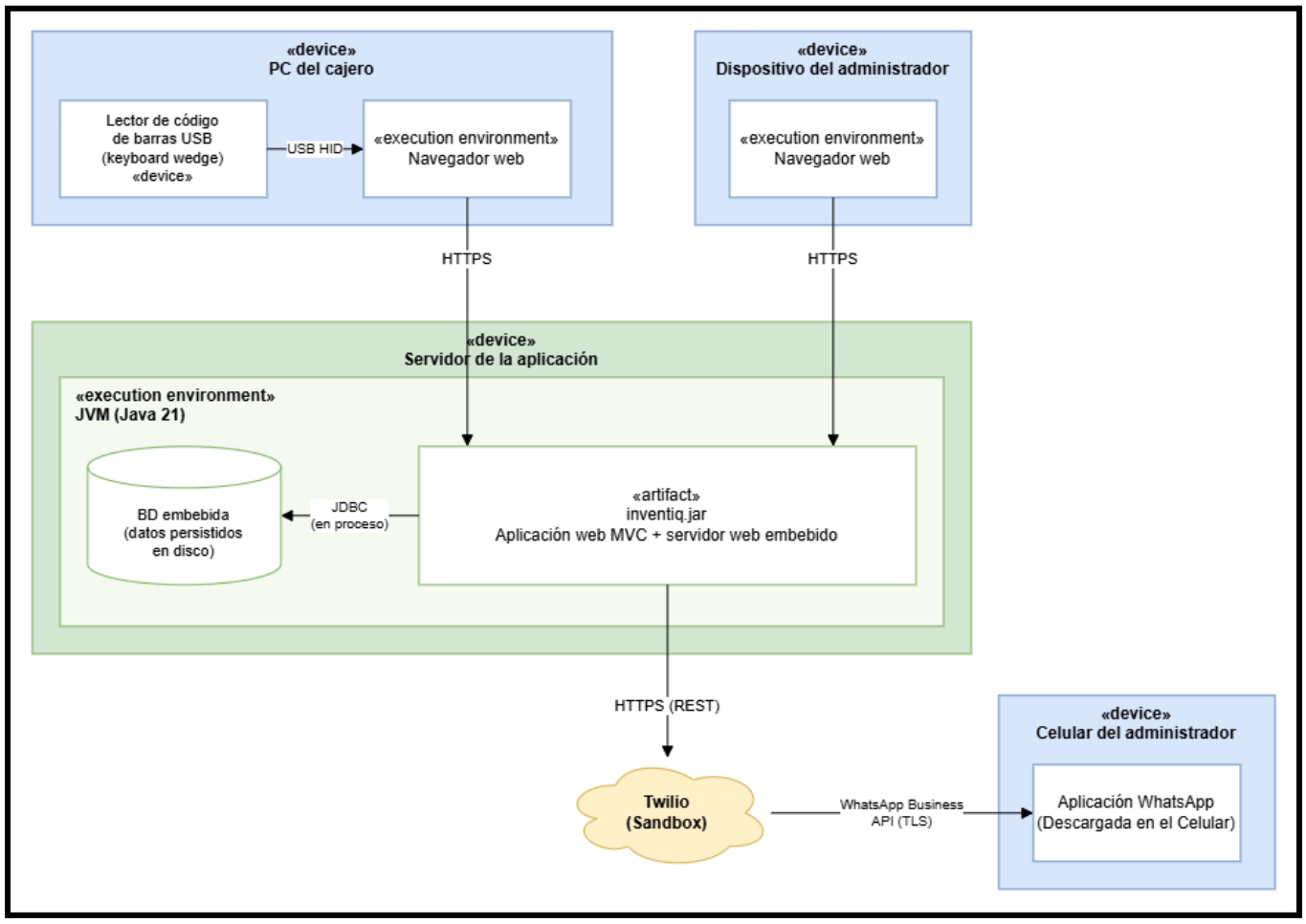


Diagram Explanation:

The system is deployed as a single unit on a single server on the provider side, which fulfills the portability requirement (NFR-05).

Clients. The cashier and the administrator access the system using only a web browser, with nothing to install. Both communicate with the application over HTTPS. The USB barcode scanner works as a keyboard (keyboard wedge) and sends the code to the cashier's browser over USB HID, so it requires no additional drivers.

Application server. A single process on the JVM (Java 21) runs the inventiq.jar file, which includes the web application with its spring boot embedded web server. The embedded database runs within the same process, and the application accesses it via in-process JDBC, with no network connections. The DB stores its data on disk so that it persists across restarts. It runs with a single command and requires no preinstalled servers.

External services. The only external dependency is Twilio. The application invokes it over HTTPS (REST), and Twilio delivers the message to the administrator's WhatsApp account through the WhatsApp Business API over TLS. In the context of the course, the Twilio sandbox is used; in production, only the subscription and the configuration would change, not the code.

Deployment assumptions. The server is hosted by the provider with in one disk because of the scholar aspect of the project, HTTPS and the data base files will be backed up periodically.

4. Requirements Traceability Matrix

Module	Module Name	Scope and responsibilities
01	Security and Access	Manages authentication, session security, and role-based access control to protect administrative functions.
02	POS and Inventory	Processes sales in real time with a barcode scanner and deducts inventory by batch applying the FIFO rule.
03	Replenishment Engine	Algorithmic core that calculates the Reorder Point (ROP), groups products by supplier, optimizes the Minimum Order (MOQ), and prioritizes purchases according to the cash budget
04	Expirations and Pricing	Monitors the shelf life of batches and calculates progressive discount scales for products close to expiring according to their sales velocity
05	Purchasing and Administration	Allows adjusting operational parameters and manual stock levels, confirming receipt of orders, generating structured messages for suppliers, and recording audits of decisions.
06	Infrastructure and Deployment	Packaging of the system as a self-contained web service (Java 21), embedded database, versioned migration scripts, and single-command run configuration, with no preinstalled dependencies.

Requirement ID	Short Description	Implementing Component / Module
FR-01	User authentication and role-based access	MOD-01: Security and Access
FR-02	Barcode checkout and FIFO deduction	MOD-02: POS and Inventory
FR-03	Automatic Reorder Point (ROP) calculation	MOD-03: Replenishment Engine
FR-04	Grouping by supplier and draft orders	MOD-03: Replenishment Engine
FR-05	Minimum Order Quantity (MOQ) optimization	MOD-03: Replenishment Engine
FR-06	Purchase prioritization under cash budget	MOD-03: Replenishment Engine
FR-07	Dynamic discounts by expiration date	MOD-04: Expirations and Pricing
FR-08	Manual adjustment of parameters and stock	MOD-05: Purchasing and Administration
FR-09	Structured message proposal for supplier	MOD-05: Purchasing and Administration
FR-10	Receipt confirmation and stock update	MOD-05: Purchasing and Administration
FR-11	Audit log of automated decisions	MOD-05: Purchasing and Administration
NFR-01	Performance efficiency (Response $< 1.5s$)	MOD-03: Replenishment Engine
NFR-02	Usability (Approval in ≤ 3 clicks)	MOD-05: Purchasing and Administration
NFR-03	Security (100% of endpoints protected)	MOD-01: Security and Access
NFR-04	Maintainability (Test coverage >= 80%)	MOD-03: Replenishment Engine
NFR-05	Portability	MOD-06: Infrastructure and Deployment

5. Risks and Assumptions

Type	Description	Mitigation / Notes
Risk	Service interruptions or latency in the Twilio API. By depending on a third-party provider for messaging, there is a risk that outages on their servers, changes to their connection policies, or latency spikes may delay or prevent the sending of the system's critical alerts.	Implement a fault-tolerance pattern so that Twilio failures do not block the main execution flow of the system. Unsent notifications must be recorded in the logs and reflected as alerts on the dashboard, whether or not the message was sent successfully.
Risk	Data inconsistencies due to transactional concurrency. There is a risk that, under a high volume of consecutive sales, multiple processes attempt to deduct the inventory of the same product at nearby moments, generating negative balances or corrupted records if the database is not handled correctly.	Implement optimistic locking at the database level for inventory records. Validate the integrity of the system by running concurrent load and stress tests before production deployments.
Risk	Incompatibility of hardware and peripheral drivers. There is a possibility that the barcode scanners acquired by the end customer use proprietary encodings or require specific drivers that do not work by default as standard input devices, that is, as a keyboard.	Design the user interface to listen for generic text input events, making the software agnostic to the physical hardware. Additionally, always keep the option of manually entering the alphanumeric code enabled and visible to guarantee operational continuity if the peripheral fails.
Risk	Database schema corruption by automatic tools. The interaction between data migration managers and automatic schema generation tools can cause unplanned destructive modifications to the structure of the database during a system restart.	Strictly disable automatic schema modification in production environments. Centralize the evolution of the database solely and exclusively through versioned migration scripts, and automate deployment to avoid error-prone manual interventions.
Assumption	The business operates at a single point of sale/warehouse, not at multiple branches.	If the real business interviewed had several locations, the current data model would have to be extended significantly; this is outside the agreed scope and must be communicated as an explicit limitation to the interviewed stakeholder.
Assumption	Suppliers have reasonably stable lead times that are known in advance (they can be defined as a fixed number of days per supplier).	If lead times vary greatly from one order to another, the proposed calculation loses accuracy, since lead time days are meant to be treated as a constant and not as a variable with its own uncertainty.
Assumption	The cashier can enter the product code either with a physical scanner or by typing it manually, and this	If the business uses products without a standardized barcode, the sales recording flow, and therefore the entire data chain that feeds reordering, could not run as designed. An alternative product identification

Type	Description	Mitigation / Notes
	code always corresponds to a single product in the catalog.	mechanism, outside the scope, would have to be defined.
Assumption	The store administrator has a device with an active WhatsApp account and can complete the process of joining the Twilio Sandbox	If this is not possible, the notification channel becomes inoperative, but the system remains fully functional through the dashboard; the adapter-based design prevents this from being a single point of failure for the entire product.

6. Conclusions and Next Steps

Main decisions. We designed InventIQ as a layered monolithic architecture with the MVC pattern, running as a single Spring Boot application (Java 21) with an embedded web server and an embedded database: one repository, one deployable artifact, and one process, with views rendered on the server. We keep the business rules (FIFO deduction, reorder point, minimum order, budget, and discounts) separate from the rest of the code, so we can unit-test them without a database or a server. The WhatsApp service (Twilio) sits behind an interface, so if it fails, the rest of the system keeps working.

Service model. InventIQ runs on the provider side. The store will only need a web browser (and optionally a USB barcode scanner that works like a keyboard), and nothing is installed on its computers. Deciding where to host the application is one of our next steps.

Trade-offs. Since the stack is fixed by the course, we depend on Java and Spring Boot. We also depend on Twilio as a third party which might bring its own problems and we reduce that risk with an adapter and with alerts on the dashboard.

How we will work. We will work in iterations, and each one will end with software that runs and that we can show to the store owner we interviewed. We use Git from day one, with each member committing under their own identity. Nothing goes directly to the main branch: every change enters through a pull request approved by another team member. We will also keep a README that lets someone outside the team clone the repository and start the application without our help.

First sprint. We want to have the basics working from start to finish:

- Create the Spring Boot project, make it start with a single command, set up the database with the initial tables, write the README, and configure the coverage report.
- Login with two roles, administrator and cashier (FR-01).
- Register a sale by typing or scanning the code, deducting stock from the corresponding batch, with unit tests, including the case of a non-existent barcode (FR-02).
- A small test sending one message through the Twilio sandbox, to confirm early that it works.

The monolithic architecture also fits our iterative and incremental process, since the whole system is one deployable unit, every sprint can end with a working, demonstrable version of the application.

7. Appendix: Glossary & References

Term	Definition
MOQ	Minimum Order Quantity: Minimum threshold of units or monetary amount required by a supplier to process a purchase order and grant benefits such as free freight.
POS	Point of Sale: Software and hardware interface where sales are processed and charged in real time through barcode scanning and direct inventory deduction
ROP	Reorder Point: Critical inventory level that triggers the automatic suggestion of a purchase order, calculated from the daily sales velocity, the lead time, and the safety stock.
FIFO	First In, First Out: Inventory management method where the oldest batches of products received are sold or consumed first to avoid expirations or obsolescence.